

PA01: Processes and Threads [31 points]

Deadline: **January 23, 2026**

Important Note:

- Use of AI to generate code is allowed. However, you should be able to understand every line of code, and answer during your Viva.
- **If AI is used for any aspect, you need to declare the components that are AI-generated. Failure to do so will result in a plagiarism penalty.**
- The code should be modular, well-commented, and structured (well-aligned). This non-functional aspect carries marks.
- Create a public github repo to add your GRS assignments.
- Read the deliverables and assignment statement carefully before starting the assignment.

(A) Write two **C** programs: **[2+2]**

1. Program A: creates 2 processes using fork()
2. Program B: creates 2 threads using pthread

(B) Write three **C** worker functions, “cpu”, “mem”, and “io”, as defined below. The processes, and threads in program A and program B should be able to execute these functions. **[2+2+2]**
Each worker function runs a loop with the count as the **last digit of your rollno $\times 10^3$** (If your last digit is “0”, use “9”)

1. Function “cpu”: CPU-intensive
 - Definition: These programs spend the majority of their execution time performing calculations and processing instructions on the CPU, rather than waiting for data from other resources.
 - Examples: complex mathematical calculations (like calculating digits of pi), video encoding, and scientific simulations
2. Function “mem”: Memory-intensive
 - Definition: These programs are bottlenecked by the speed and capacity of the system's memory (RAM) subsystem. They require large amounts of data to be moved between the CPU and memory, or they process data that is too large to fit efficiently within the CPU's internal cache.
 - Examples: large-scale data sorting/processing, and in-memory databases.
3. Function “io”: I/O-intensive
 - Definition: These programs spend most of their time waiting for input or output operations to complete, during which time the CPU often sits idle or switches to other tasks.
 - Examples: reading/writing large files to disk, database queries, network requests and user-interactive programs that wait for keyboard input.

(C) For all the six combinations of program+worker, i.e., A+cpu, A+mem, A+io, B+cpu, B+mem, B+io, do the following:

- You will observe the system using:
 - **top** : record CPU% consumed by the program. Use [taskset](#) to pin your program to the specific set of CPUs.
 - **iotop** : to observe disk statistics
 - Measure execution time using: **time ./<program_name> <args>**
- Write a bash script that **automates** the process of **[1+3+2+3]**
 - executing the six program variants
 - For each program, automate the measurement collection (Hint: you may have to run the command such as “top” after every second, and summarize the final observation)
 - Generate the final output in the following format for each variant:
Program+Function CPU% Mem IO
 - Analyze and discuss your observations

(D) Given (C)

1. increment **[1+1]**
 - a. The number of processes in Program A, say, 2, 3, 4, 5
 - b. The number of threads in Program B, say, 2, 3, 4, 5, 6, 7, 8
2. Amend the bash script in (C) to **[1+2+4]**
 - a. **Automate** (D) (1) execution
 - b. (Automate) Plot the measured metrics in a way that makes sense to analyse the results. Use your creativity.
 - c. Analyze and discuss your observations

Deliverables:

- Add the bulleted items to a folder named <roll_num>_PA01, and upload the compressed file, <roll_num>_PA01.zipSource code (e.g., C code, bash scripts, etc.)
 - A Makefile for compiling the program. **[1]**
 - Raw CSV data containing the measurements **[1]**
 - Plots (PDF/PNG)
 - Documentation (README) explaining the implementation and usage. **[1]**
 - Report comprising (a) screenshots, plots, and analysis for each question, (b) AI usage declaration, and (c) Your github repo URL
- Upload the above to the github repo with the folder name “GRS_PA01”(not the zip, but the folder)

Note –

1. For part A, if your roll number's last digit is 0, then use 9.
2. For part A, parent process (in code A) and main function (in code B) are not considered. It says 2 processes for (i) **means 2 child processes apart from the parent** and for (ii) **2 threads apart from main()**.
3. There should only be a **single report PDF**. No doc/docx is allowed.
4. There would be **2 CSVs** - 1 for part C and 1 for part D.
5. **Naming convention of files:**
 - (i) **Every** file will **start** with your **roll no. followed by part then followed by deliverable** like *MT25xxx_Part_C_CSV.csv*, *MT25xxx_Part_A_Program_A.c*, *MT25xxx_Report.pdf*.
 - (ii) If in a given part there are many files, then name prefix must be *MT25xxx_Part_<Name of part>_<file name>.extension*, like *MT25xxx_Part_B_XYZ.c*, *MT25xxx_Part_B_PQR.c*, *MT25xxx_Part_C_shell.sh*
 - (iii) These conventions **don't apply** to Readme and Makefile. Use **standard** naming conventions for these 2 files. Do mention your roll no. **at the top inside** the Readme file.

Strictly adhere to the naming conventions, else heavy penalties will be levied.
6. Do not upload binaries or executable files either in the zip folder to be submitted in google classroom or on the github. **0 marks will be awarded if these files are submitted anywhere.**
7. There should be a single zipped folder, as told earlier, having all the files. There shouldn't be any subfolder inside the main folder. **The main folder will consist of the files belonging to all 4 parts, Readme, Makefile, report.**